Website Status Checker

A guided tour of the concepts, functions, features and details

Python standard library only | purely procedural design | Levels 1 to 4

Read it alongside website_status_checker_annotated.py

The big picture

what the program is for

You give the program a list of website addresses. It visits each one, records whether the site is UP or DOWN and how fast it answered, prints the results, and saves them to files so you can track reliability over time.

Input cleaning

Reads URLs from a file or the command line, skips comments, rejects bad URLs, removes duplicates.

Concurrent checking

A thread pool visits many sites at the same time, so one slow site does not delay the rest.

Resilient probing

Timeouts, retries with a delay, and exact error labels: DNS, timeout, SSL, HTTP status.

Reports and history

A CSV report each run, optional JSON, and a history log that grows with every run.

Watch mode

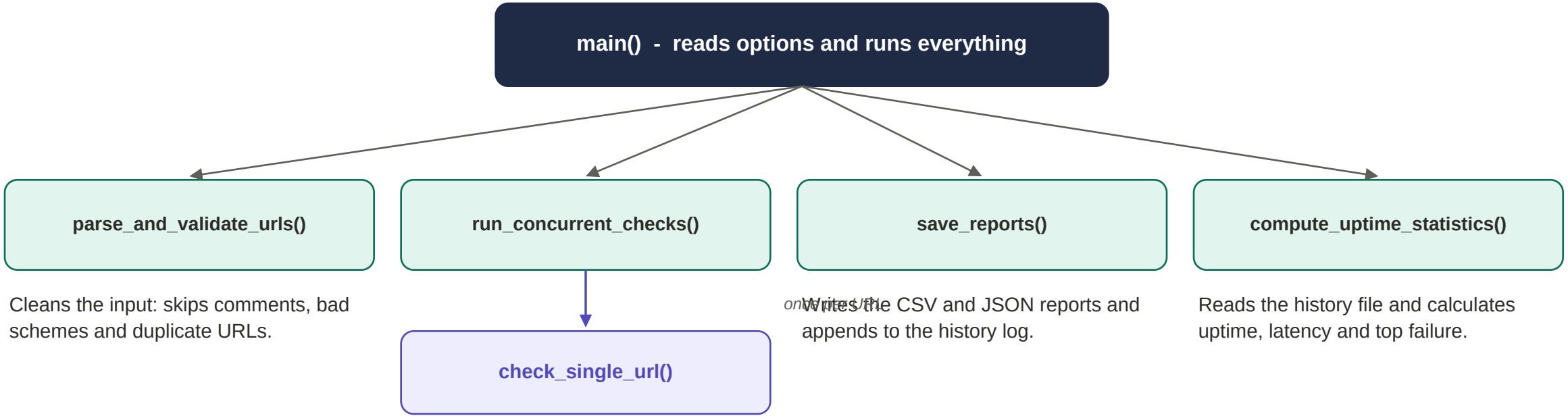
Repeats the checks every N seconds and raises an alert when a site flips between UP and DOWN.

Stats mode

Reads the history and reports uptime percentage, average latency and the most common failure.

Architecture: six functions, one pipeline

decomposition in action



Data flow

Command-line options

set up with argparse

Option	Type	Default	What it does
`urls`	list	none	URLs typed directly after the script name
`-f, --file`	path	none	Text file of URLs, one per line
`-t, --timeout`	int	5	Seconds to wait on each attempt
`-r, --retries`	int	1	Total attempts allowed per URL
`-d, --delay`	float	1.0	Seconds to pause between attempts
`--allow-redirects`	switch	off	Count a 3xx answer as UP
`-w, --workers`	int	5	Number of parallel threads
`--report`	text	status_report.csv	CSV report filename
`--json-report`	text	none	Optional JSON report filename
`--history`	text	status_history.csv	Log that grows across runs
`--watch`	int	none	Repeat every N seconds until Ctrl+C
`--stats`	switch	off	Print uptime statistics from history, then exit

A switch is an on/off flag (action="store_true"): present means True, absent means False.

Function 1: parse_and_validate_urls()

collect and clean the input

1. **Read the file**
   Skip blank lines and lines that start with #. Keep each URL with its line number so warnings can point to it.

2. **Add typed URLs**
   URLs given on the command line are added with the source label "CLI Argument".

3. **Validate**
   urlparse() splits the URL. The scheme must be http or https, and a host (netloc) must exist.

4. **Deduplicate**
   A set remembers URLs already accepted, so repeats are skipped with a warning.

Python concepts here

- **with open(...)** closes the file automatically.
- **enumerate(f, 1)** gives line numbers from 1.
- **Tuples** store (url, source) pairs.
- **set** fast "have I seen this?" checks.
- **continue** skips to the next item.
- **sys.exit(1)** stops the program with an error code.

```
parsed = urllib.parse.urlparse(url)
if parsed.scheme not in ('http', 'https') or not parsed.netloc:
    continue          # malformed or unsupported: skip it
```

Output: a clean list of unique, valid URLs. Warnings go to the error stream (stderr), not normal output.

Function 2: check_single_url()

probe one website

Inputs

- **url** the address to probe
- **timeout=5** seconds to wait per attempt
- **retries=1** total attempts allowed
- **delay=1** pause between attempts
- **allow_redirects** count 3xx as UP when True

State it tracks

- **status_result** starts as "DOWN"
- **status_code** None until a reply arrives
- **response_time** stopwatch reading
- **error_type** None means no error
- **attempts** tries used so far
- **final_url** detects redirects

Record it returns

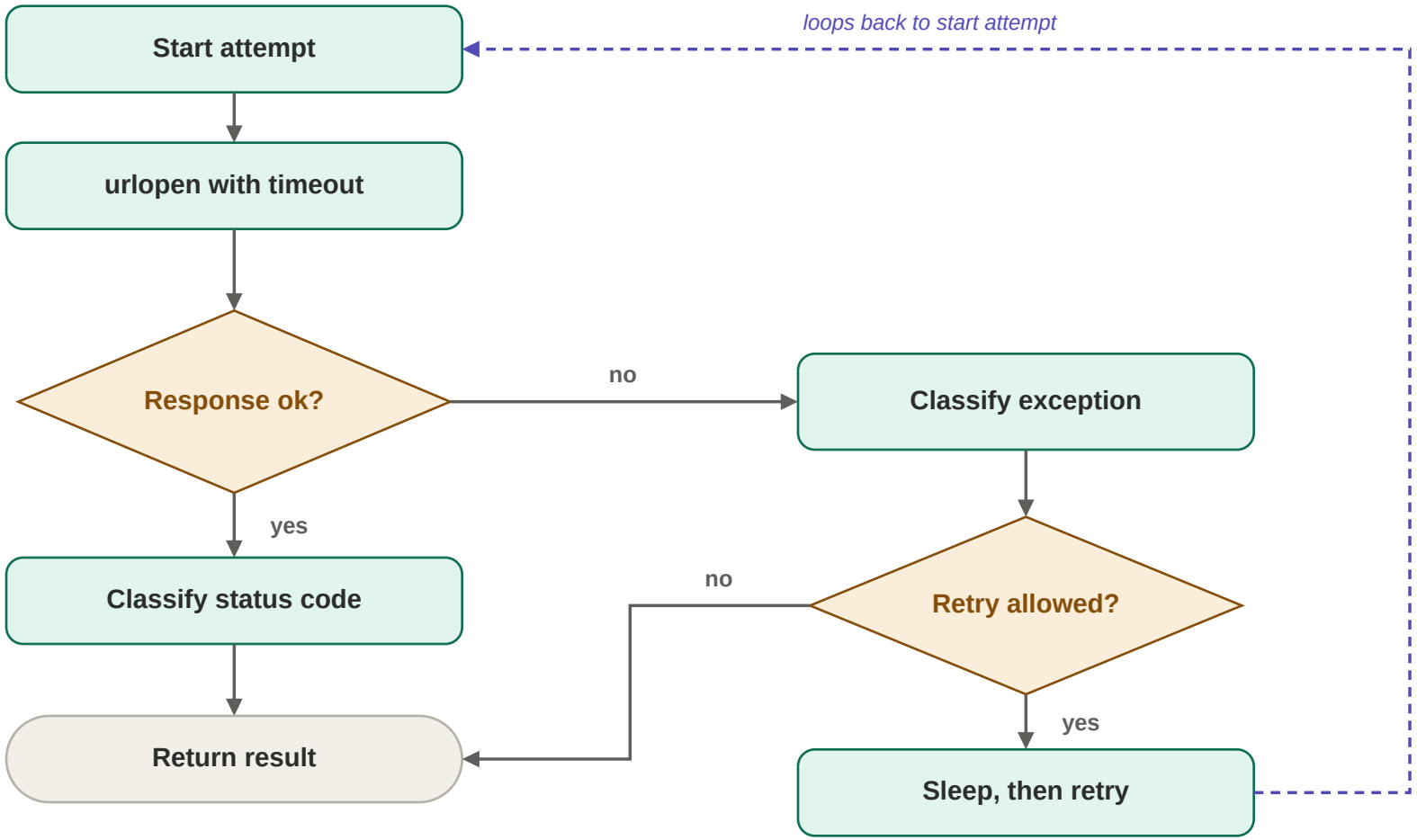
```
url
status
status_code
response_time
attempts
redirect_target
redirected
error_type
timestamp
```

Key idea: a pessimistic default

A site is DOWN until a good response proves it UP. Every outcome is written into the same few state variables, then returned as one dictionary, exactly like the state tracker in Week 1.

The retry logic

decisions are diamonds



Retry rule

- **Retries:** DNS errors and timeouts only, while attempts remain.
- **Stops at once:** HTTP error codes, SSL errors, unknown errors.
- **Why:** a 404 or a bad certificate will not fix itself on a retry.

How failures are classified

exception to label

What happened	Exception caught	Label recorded	Retry?
Server replied with an error code	HTTPError	HTTPError_404 (code kept)	No
Domain name could not be resolved	URLError (gaierror)	DNS error	Yes
Site took too long to answer	URLError or socket.timeout	TimeoutError	Yes
Certificate or encryption problem	ssl.SSLError	SSLError	No
Other transport problem	URLError (other)	URLError_<type>	Yes
Reply received but not a 2xx	none raised	HTTPStatus_<code>	No
Anything unexpected	Exception	Exception_<type>	No

Order matters

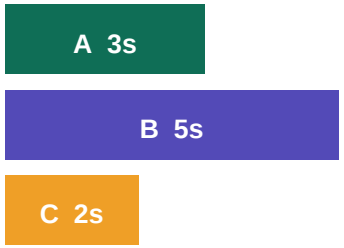
HTTPError is a special kind of URLError. If URLError were listed first it would catch HTTP errors too, so the most specific except block always goes first.

Function 3: run_concurrent_checks()

many sites at once

One after another: 10 seconds in total

Thread pool, all at once: 5 seconds in total



Total time is the slowest site, not the sum of all sites.

Why threads suit this job

The program spends almost all its time waiting for the network, not computing. While one thread waits for a reply, the others keep working.

1. **Create the team**
   ThreadPoolExecutor(max_workers=N) starts N workers.
2. **Hand out jobs**
   submit() starts check_single_url for a URL and returns a future, a ticket for the result. A dictionary stores {future: url}.
3. **Collect as they finish**
   as_completed() yields tickets fastest-first. result() returns the record, or an exception if the worker crashed.
4. **Survive crashes**
   A crashed worker becomes a DOWN record with a ThreadException label, so no URL is ever lost.
5. **Restore the order**
   Results finish in random order, so they are sorted back to the order the URLs were listed (a lambda sort key).

Function 4: save_reports()

files that outlive the run

Output	File mode	Behaviour	Why it is built this way
CSV report	'w' overwrite	Header row, then one row per URL	A fresh snapshot of the latest run
JSON report	'w' overwrite	Whole result list, indent=4, only if requested	Easy for other programs to read
History log	'a' append	Header written only when the file is new	Builds trend data across many runs

csv.DictWriter

Takes dictionaries and writes each one as a row. The fieldnames list fixes the column order, and newline='' stops blank rows appearing on Windows.

The nine columns

```
url, status, status_code
response_time, attempts
redirect_target, redirected
error_type, timestamp
```

Check file_exists BEFORE opening in append mode, or you cannot tell whether the header is needed.

Function 5: compute_uptime_statistics()

turning history into insight

```
stats = defaultdict(lambda: {
    "total": 0,
    "up": 0,
    "response_times": [],
    "failures": defaultdict(int)
})

# stats[url]["total"] += 1
# a new URL gets a fresh record
# automatically - no key check needed
```

defaultdict builds a missing entry on first use, so the loop never asks "does this key exist yet?".

Uptime %

`up / total x 100`

Average latency

`sum(times) / len(times)  (UP checks only)`

Top failure

`max over failures, compared by count`

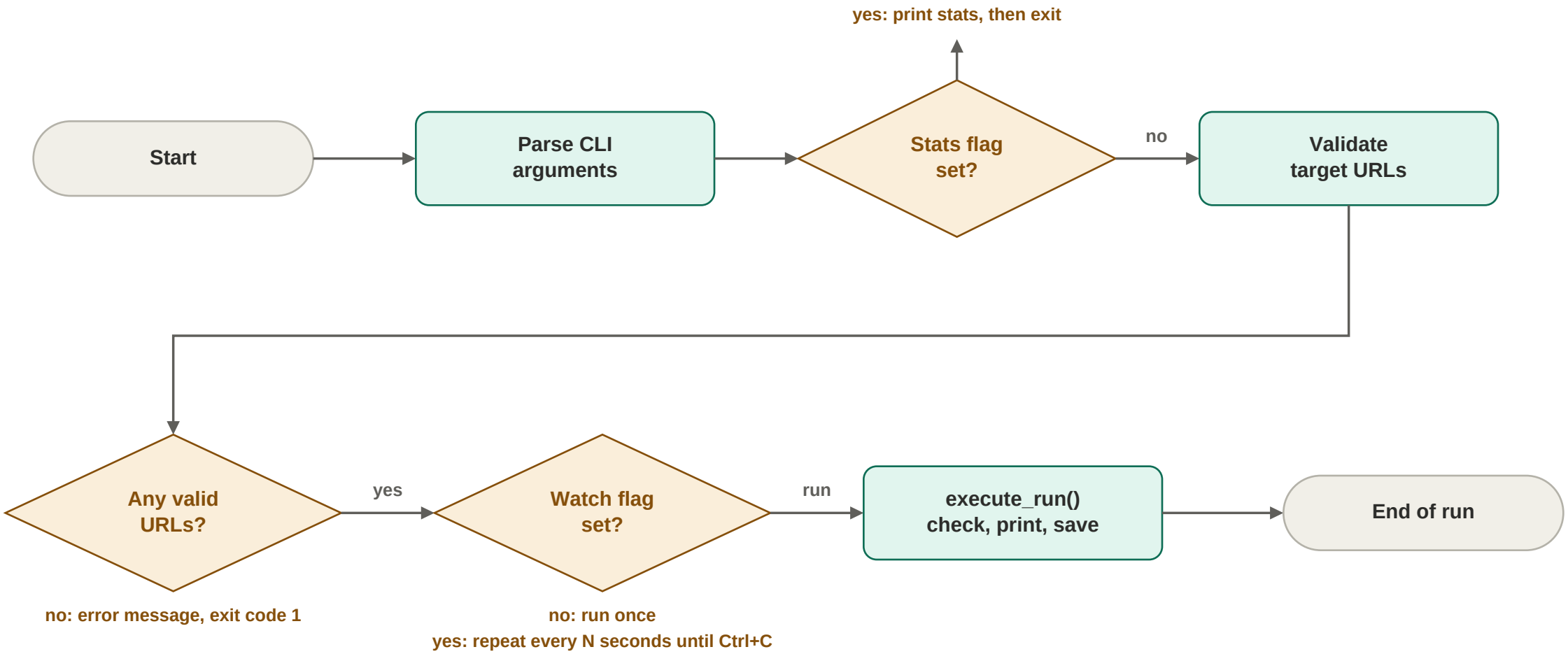
Slowest URL

Concepts and safeguards

- **csv.DictReader:** each row becomes a dictionary.
- **float():** CSV values are text, so convert before maths.
- **try / except ValueError:** a bad number is ignored, not fatal.
- **Division guard:** "if total > 0" prevents dividing by zero.
- **lambda x: x[1]:** tells max() to compare (error, count) pairs by count.
- **Returns a tuple:** (summary list, slowest URL).

Function 6: main() control flow

decisions are diamonds



Python concepts used, and where you met them

Concept	Where it appears in the project	Link to your course
Functions	Six functions, one job each	Decomposition; is_prime kept separate
for vs while	Retry uses for; watch mode uses while True	Week 2: known vs unknown repeat count
Dictionaries	Result records, stats, previous_states	Week 3: list of dictionaries
Sets	Fast duplicate detection	Membership checks, like the in keyword
try / except	Network error handling	Planned error-handling week
Scope	execute_run reads main()'s variables	Global vs local variables
lambda	Sort key and max key	Optional sorting idea for inventory
File modes w / a	Overwrite report, append history	Overwrite vs adjust, as in stock_count
Mutability	previous_states updated in place	Lists and dictionaries are mutable

Gotchas and design decisions

details worth knowing

- **Pessimistic default.** Status starts DOWN and only a good response flips it to UP.
- **Specific errors first.** HTTPError must be caught before its parent URLError.
- **Redirects are usually followed automatically.** urlopen follows most redirects itself, so the final status is normally 2xx and the 3xx branch rarely runs. A redirect is detected by comparing the final URL with the requested one.
- **Timeout is per attempt.** The worst case for one URL is roughly retries x timeout plus the delays between tries.
- **Random finishing order.** as_completed returns fastest first, so results are re-sorted to match the input list.
- **History columns are fixed.** If the column list ever changes, older rows in the append-only history no longer match its header.
- **Warnings use stderr.** Normal results and warnings are separate streams, so output can be redirected cleanly.

Try it, then test yourself

Run it (save the script as checker.py)

```
python checker.py https://example.com

python checker.py -f sites.txt -t 3 -r 2

python checker.py -f sites.txt --watch 60

python checker.py --stats
```

Put one URL per line in sites.txt. Lines starting with # are ignored.

Explain these in plain words

- **1.** Why is a site DOWN until proven UP?
- **2.** Why must HTTPError come before URLError?
- **3.** Why are results re-sorted after as_completed?
- **4.** Why is the CSV overwritten but the history appended?
- **5.** Which errors retry, and why not the others?
- **6.** Why while True for watch mode, but for for retries?